**Assignment Code: DS-AG-033**

# VectorDB, Hugging Face & Ollama | Assignment

**Instructions:** Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

**Total Marks**: 200

**Question 1:** What is a Vector Database (VectorDB) and how is it different from traditional databases?

**Answer:**

A **Vector Database (VectorDB)** is a specialized database designed to store, manage, and search high-dimensional vector embeddings generated by machine learning models. These embeddings represent data such as text, images, audio, or video in numerical form.

Vector databases are mainly used in **AI applications like semantic search, recommendation systems, and Retrieval-Augmented Generation (RAG).**

**Differences between Vector Databases and Traditional Databases**

| Feature | Vector Database | Traditional Database |
|---|---|---|
| Data Type | Stores vector embeddings | Stores structured data (tables, rows) |
| Search Method | Similarity search (cosine similarity, Euclidean distance) | Exact match queries |
| Use Case | AI/ML, semantic search, recommendation | Business data, transactions |
| Query Type | Finds most similar vectors | Uses SQL queries |
| Data Representation | High-dimensional vectors | Structured data like text, numbers |

**Question 2:** Explain the various types of VectorDBs available and describe their suitability for different use cases.

**Answer:**

Vector databases can be categorized into several types depending on their architecture and usage.

**1. Open Source Vector Databases**

Examples: ChromaDB, Milvus, FAISS

**Features**

- Free and customizable
- Suitable for research and small-scale applications
- Can run locally

**Use cases**

- AI experimentation
- Academic research
- Small AI projects

**2. Cloud-based Vector Databases**

Examples: Pinecone, Weaviate Cloud

**Features**

- Managed infrastructure
- Scalable and reliable
- Easy integration with AI applications

**Use cases**

- Large-scale AI systems
- Production deployments
- Enterprise applications

**3. Hybrid Vector Databases**

Examples: ElasticSearch with vector search

**Features**

- Combine traditional search with vector search
- Support both keyword and semantic search

**Use cases**

- Document search engines
- E-commerce product search
- Knowledge retrieval systems

**Question 3:** Why is Chroma DB considered important in the context of AI/ML projects? Describe its key features.

**Answer:**

ChromaDB is a lightweight open-source **vector database designed specifically for AI applications**. It helps developers store embeddings and perform fast similarity searches.

**Importance in AI/ML**

- Supports **semantic search**
- Used in **Retrieval-Augmented Generation (RAG)**
- Easy integration with **LLMs and LangChain**

**Key Features**

1. **Embedding Storage**
   Stores embeddings generated from text, images, or documents.
2. **Similarity Search**
   Finds the most relevant documents based on vector similarity.
3. **Lightweight and Fast**
   Can run locally without heavy infrastructure.

4. **Easy Integration**
   Works well with Python libraries like:

- LangChain
- Hugging Face
- OpenAI

5. **Persistent Storage**
   Allows saving embedding for reuse.

**Question 4:** What are the benefits of using Hugging Face Hub for generative AI tasks?

**Answer:**

Hugging Face Hub is a platform that provides **thousands of pre-trained AI models and datasets** for developers.

**Benefits**

**1. Large Model Repository**

Provides access to thousands of models for:

- Text generation
- Image generation
- Speech recognition
- Translation

**2. Easy Model Sharing**

Developers can upload and share their models with the community.

**3. Pre-trained Models**

Models are already trained, saving **time and computational resources**.

**4. Easy Integration**

Works easily with Python libraries such as:

- Transformers
- Datasets
- Accelerate

**5. Collaboration**

Teams can collaborate on AI projects and improve models together.

**Question 5:** Describe the process and advantages of navigating and using pre-trained models from the Hugging Face Hub.

**Answer:**

**Process of Using Pre-trained Models**

**Step 1: Install Transformers**

```
pip install transformers
```

**Step 2: Import the pipeline**

```
from transformers import pipeline
```

**Step 3: Load a model**

```
generator = pipeline("text-generation", model="gpt2")
```

**Step 4: Generate text**

```
output = generator("Artificial Intelligence is", max_length=30)
print(output)
```

**Advantages**

1. **Time Saving**
   No need to train models from scratch.
2. **High Accuracy**
   Pre-trained on large datasets.
3. **Easy to Use**
   Only a few lines of code required.
4. **Wide Variety of Models**
   Models available for many AI tasks.

**Question 6:** Install and set up Chroma DB, and insert sample vector data for semantic search.

(*Include your Python code and output in the code box below.*)
**Answer:**

**Code:**

```
import chromadb

# Create client
client = chromadb.Client()

# Create collection
collection = client.create_collection(name="my_documents")

# Add sample documents
collection.add(
    documents=[
        "Machine learning is a field of AI",
        "Deep learning uses neural networks",
        "Python is widely used for data science"
    ],
    ids=["1", "2", "3"]
)

# Query the collection
results = collection.query(
    query_texts=["What is AI?"],
    n_results=2
)

print(results)
```

**Question 7**: Demonstrate how to download and fine-tune a Hugging Face model for a text generation task.

(*Include your Python code and output in the code box below.*)

**Answer:**

**Code:**

```
from transformers import AutoTokenizer, AutoModelForCausalLM
from transformers import Trainer, TrainingArguments
from datasets import load_dataset

# Load dataset
dataset = load_dataset("wikitext", "wikitext-2-raw-v1")
```

```
# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained("gpt2")

# Load model
model = AutoModelForCausalLM.from_pretrained("gpt2")

# Tokenize dataset
def tokenize(example):
    return tokenizer(example["text"], truncation=True)

tokenized_dataset = dataset.map(tokenize, batched=True)

# Training arguments
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=2,
    num_train_epochs=1
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"]
)

trainer.train()
```

**Question 8**: Create a custom LLM using Ollama and Llama2, and run it locally for basic text prompts.

(*Include your Python code and output in the code box below.*)

**Answer:**

**Code:**

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

result = generator("Explain Artificial Intelligence", max_length=50)

print(result)
```

**Question 9**: Implement a basic RAG (Retrieval-Augmented Generation) system using Ollama with Llama3.

(*Include your Python code and output in the code box below.*)

**Answer:**

**Code:**

```
from langchain.llms import Ollama
from langchain.vectorstores import Chroma
from langchain.embeddings import HuggingFaceEmbeddings

# Load embedding model
embedding = HuggingFaceEmbeddings()

# Create vector database
vectordb = Chroma(persist_directory="db", embedding_function=embedding)

# Load LLM
llm = Ollama(model="llama3")

# Query system
query = "What is machine learning?"

docs = vectordb.similarity_search(query)

response = llm.invoke(query + str(docs))

print(response)
```

**Question 10:** A health-tech startup wants to build a chatbot that can answer user queries based on medical research articles. Propose and explain a solution using Hugging Face models for understanding, VectorDB for retrieval, and Ollama for generation.

(*Include your Python code and output in the code box below.*)

**Answer:**

A health-tech startup can build an intelligent chatbot using the following architecture.

**Step 1: Data Collection**

Collect medical research articles, journals, and health documents.

**Step 2: Text Processing**

Use Hugging Face models to convert text into embeddings.

Example model:

- Sentence Transformers
- BioBERT

**Step 3: Vector Storage**

Store embeddings in a **Vector Database** such as:

- ChromaDB
- Pinecone
- FAISS

This allows fast semantic search of medical documents.

**Step 4: Retrieval System**

When a user asks a question:

1. Convert the question into embeddings
2. Search similar documents in the VectorDB

3. Retrieve relevant medical information

**Step 5: Response Generation**

Use Ollama with a local LLM such as **Llama3** to generate a natural response based on retrieved documents